

Лабораторная работа №2

«Понятие репозитория проекта. Структура проекта»

Время на выполнение:

4 часа

Дисциплина:

«Инструментальные средства разработки программного обеспечения»

Формируемые компетенции:

ПК 2.2, ПК 2.5, ОК 2, ОК 4, ОК 5

Цель работы

Получение практических навыков создания и организации репозитория программного проекта, выбора рациональной структуры каталогов, работы с системой контроля версий Git (и её российскими аналогами), а также оформления проектной документации.

Задачи работы

- Изучить понятие репозитория, основные функции систем контроля версий.
- Освоить типовую структуру программного проекта.
- Научиться создавать локальный и удалённый репозиторий.
- Освоить базовые команды Git: init, add, commit, status, log, branch, merge.
- Настроить файл .gitignore для исключения временных и служебных файлов.
- Оформить файл README.md с описанием проекта.
- Ознакомиться с российскими аналогами Git, включёнными в Единый реестр российского ПО.
- (Для повышенного уровня) освоить работу с ветками, удалённым репозиторием и простейшими средствами непрерывной интеграции.

1. Краткая теоретическая часть

1.1. Понятие репозитория

Репозиторий — это хранилище файлов проекта и истории их изменений. В современных системах контроля версий (Git, Mercurial, Subversion) репозиторий позволяет:

- хранить несколько версий одного проекта;
- возвращаться к предыдущим версиям;
- работать над проектом параллельно нескольким разработчикам;
- отслеживать авторство изменений.

Git — распределённая система контроля версий, наиболее распространённая в индустрии. Основные понятия:

- рабочая копия — файлы проекта на диске;
- индекс (staging area) — промежуточная область для подготовки коммита;
- коммит (commit) — зафиксированное состояние проекта;
- ветка (branch) — независимая линия разработки;
- удалённый репозиторий (remote) — серверная копия проекта (GitHub, GitLab, Bitbucket).

Важно! Все команды Git вводятся в терминале (командной строке) вашей операционной системы:

Windows: используйте Git Bash (устанавливается вместе с Git) или командную строку (cmd), предварительно добавив Git в PATH. Рекомендуется именно Git Bash, так как он эмулирует среду Linux и поддерживает все команды.

Linux: откройте терминал (например, сочетанием клавиш Ctrl+Alt+T).

macOS: откройте Терминал (Applications → Utilities → Terminal).

Перед выполнением команд необходимо перейти в каталог проекта с помощью команды cd. Например:

cd /путь/к/проекту

Для Windows в Git Bash путь может выглядеть так:

cd /c/Users/Имя/Projects/MyProject

1.2. Российские аналоги Git в Едином реестре ПО

В условиях импортозамещения в России создан Единый реестр российского программного обеспечения — государственная информационная система, содержащая сведения об отечественных программных продуктах, соответствующих требованиям безопасности и рекомендованных для использования в государственных и коммерческих организациях.

В реестр включены следующие российские аналоги Git:

Продукт	Краткое описание	Номер в реестре
GitFlic	Российская DevOps-платформа для хранения кода и совместной разработки, позиционируется как альтернатива GitLab. Доступна в облачном и локальном вариантах.	№15861
GitFlame	Сервис хранения репозиторий с версионным контролем исходного кода, созданный для разработчиков и DevOps-инженеров. Построен на системе контроля версий Git.	№21590
Gitlife Enterprise	Модульная платформа для управления полным жизненным циклом проектов, включает модуль «Код» на базе Git.	№29396

Продукт	Краткое описание	Номер в реестре
Deckhouse Code	Решение для непрерывной разработки и управления жизненным циклом ПО, предоставляет инструменты для хранения кода и CI/CD.	№27871
EvaGit	Система управления Git-репозиториями, поддерживающая ветки, коммиты, пулл-реквесты и код-ревью. Входит в состав платформы EvaTeam, включённой в реестр.	(в составе EvaTeam)
Mos.Hub	Городской репозиторий ПО с открытым кодом, позиционируется как аналог GitHub для российских разработчиков.	(городской проект)

Большинство из них построены на стандарте Git, что обеспечивает совместимость и привычный рабочий процесс, а также доступны как в облачном, так и в локальном варианте развёртывания.

1.3. Структура проекта

Структура проекта — это организация каталогов и файлов, которая делает проект понятным, поддерживаемым и удобным для командной работы. Типовая структура может включать:

```

project/
├── src/           # исходный код
├── docs/          # документация (ТЗ, описание, руководства)
├── tests/         # тесты
├── build/         # артефакты сборки (обычно в .gitignore)
├── lib/           # сторонние библиотеки
├── README.md     # описание проекта
├── .gitignore     # список игнорируемых файлов
└── LICENSE       # лицензия (при необходимости)

```

.gitignore — файл, в котором перечисляются шаблоны имён файлов и каталогов, не подлежащих версионированию (например, *.exe, build/, __pycache__/).

README.md — основной документ, с которого начинается знакомство с проектом. В нём указываются: назначение, требования, инструкция по сборке и запуску, авторы.

2. Задание

На основе технического задания, разработанного в лабораторной работе №1, выполнить организацию репозитория проекта. Проект должен соответствовать выбранному варианту (например, «Решение квадратного уравнения», «Обработка матрицы» и т.п.).

Работа дифференцирована по уровням сложности. Студент может выбрать уровень самостоятельно в зависимости от того на какой балл по данной дисциплине он претендует. Оценка выставляется в соответствии с уровнем.

Все команды выполняются в терминале (Git Bash для Windows) после перехода в каталог проекта.

Уровень 1 (базовый) — 53–79 баллов

Обязательные действия:

1. Откройте терминал (Git Bash в Windows) и перейдите в каталог, где будет располагаться проект:

`cd /путь/к/будущему/проекту`

2. Создайте локальный репозиторий:

`git init`

3. Разработайте структуру каталогов проекта:

`src/` — исходный код;

`docs/` — документация;

`tests/` — тесты (могут быть пустыми на этом этапе);

`README.md` — описание проекта;

`.gitignore` — исключения.

4. Создать каталоги можно командами терминала:

`mkdir src docs tests`

5. Поместите в `docs/` файл с техническим заданием из ЛР №1 (в формате `.md` или `.docx`). Например, скопируйте его вручную или командой `cp`.

6. Создайте `README.md` с помощью текстового редактора (например, `nano README.md` или через проводник).

Содержание:

- название проекта;
- краткое назначение;
- требования к окружению;
- инструкция по запуску (предварительная);
- автор(ы).

7. Настройте `.gitignore` (создайте файл и внесите шаблоны):

- исключить временные файлы (`*.tmp`, `*.log`);
- исключить каталог сборки (`build/`, `dist/`);
- исключить файлы окружения (`.env`, `venv/`, `__pycache__/`).

8. Выполните не менее 3 коммитов с осмысленными сообщениями. Для этого после каждого изменения добавляйте файлы в индекс и фиксируйте:

`git add .`

`git commit -m "Начальная структура проекта"`
(повторите для документации и исходного кода)

9. Продемонстрируйте историю:

`git log --oneline --graph`

Отчёт должен содержать: структуру каталогов, листинг `.gitignore`, вывод `git log`, скриншоты.

Уровень 2 (продвинутый) — 80–92 балла

Дополнительно к уровню 1:

1. Создайте удалённый репозиторий на одной из платформ (на выбор):
 - Зарубежные: GitHub или GitLab (через веб-интерфейс).
 - Российские (из Единого реестра): GitFlic, GitFlame, Gitlife Enterprise, Mos.Hub.В отчёте укажите, какую платформу вы выбрали и почему.

2. Свяжите локальный репозиторий с удалённым (команды в терминале):

`git remote add origin <URL>`

`git push -u origin main`

(если основная ветка называется master, замените main на master).

3. Создайте ветку develop:

`git checkout -b develop`

4. Внесите изменения в проект (например, добавьте модуль или функцию), закоммитьте:

`git add .`

`git commit -m "Добавлен модуль ..."`

5. Выполните слияние ветки develop в main:

`git checkout main`

`git merge develop`

6. Смоделируйте и разрешите конфликт слияния:

- создайте конфликтную ситуацию (измените одну и ту же строку в двух ветках);
- выполните слияние, разрешите конфликт вручную (отредактируйте файл);

зафиксируйте результат:

`git add .`

`git commit -m "Разрешён конфликт слияния"`

7. Опишите в README.md выбранную стратегию ветвления (Git Flow, GitHub Flow или иную) и обоснуйте её.

8. Проведите сравнительный анализ выбранной платформы (зарубежной или российской) с альтернативой: укажите преимущества и недостатки, доступные функции, удобство интерфейса.

Отчёт дополняется: ссылкой на удалённый репозиторий, описанием конфликта и способа его разрешения, схемой ветвления, результатами сравнения платформ.

Уровень 3 (высокий) — 93–100 баллов

Дополнительно к уровню 2:

1. Настройте простейший CI/CD¹ с помощью GitHub Actions (или GitLab CI, или аналогичного инструмента на российской платформе, если поддерживается).
2. Создайте файл `.github/workflows/ci.yml` (для GitHub Actions). Настройте автоматический запуск:
 - установки зависимостей;
 - линтинга (например, `flake8` для Python, `cppcheck` для C++)²;
 - сборки или запуска тестов (если тесты уже есть).
3. Создайте шаблоны для Issue и Pull Request:
`.github/ISSUE_TEMPLATE/bug_report.md`;
`.github/PULL_REQUEST_TEMPLATE.md`.
4. Оформите документацию проекта в формате Markdown:
`docs/architecture.md` — описание архитектуры;
`docs/testing.md` — план тестирования.
5. Создайте тег версии (команды в терминале):
`git tag -a v0.1.0 -m "Первая версия"`
`git push origin v0.1.0`
6. Проведите ревью кода³ (самостоятельно или с одноклассником) и опишите его результаты в `docs/review.md`.
7. Подготовьте краткий обзор российских аналогов Git (не менее трёх), включённых в Единый реестр российского ПО: укажите их

¹ CI (Continuous Integration / Непрерывная интеграция): автоматическая сборка и запуск тестов каждый раз, когда разработчик добавляет новый код в репозиторий.

CD (Continuous Delivery / Непрерывная доставка или Continuous Deployment / Непрерывное развертывание): автоматическая отправка проверенного кода на тестовые или рабочие сервера.

² Линтинг — это автоматическая проверка исходного кода с помощью специальных программ (линтеров) на наличие ошибок, опечаток, потенциальных багов и нарушений стандартов стиля.

³ Код-ревью (от англ. code review — проверка кода) — это процесс, когда один или несколько разработчиков изучают и оценивают исходный код коллеги до его слияния в общую кодовую базу.

назначение, функциональные возможности, преимущества и ограничения. Оформите в виде docs/russian_git_analogs.md.

Отчёт дополняется: файлами конфигурации CI, скриншотами успешного прохождения пайплайна⁴, описанием тега, ревью и обзором российских аналогов.

3. Содержание отчёта

Отчёт должен включать:

- ✓ Титульный лист.
- ✓ Цель работы.
- ✓ Задание (с указанием выбранного уровня).
- ✓ Структуру проекта (в виде дерева каталогов).
- ✓ Листинг .gitignore.
- ✓ Листинг README.md.
- ✓ Используемые команды Git с пояснениями (указать, что команды вводились в терминале).
- ✓ Вывод git log --oneline --graph.
- ✓ Скриншоты ключевых этапов (создание репозитория, коммиты, ветки, удалённый репозиторий).

Для уровня 2: описание конфликта и его разрешения, сравнительный анализ платформ.

Для уровня 3: описание CI, ссылки на шаблоны, тег, обзор российских аналогов Git.

- ✓ Ответы на контрольные вопросы.
- ✓ Выводы.

Формат файла с отчетом: .docx.

Имя файла: №2(Уровень)_Фамилия_Имя.docx.

⁴ Пайплайн (от англ. pipeline — «трубопровод») — это четкая, пошаговая последовательность действий или процессов, которая превращает входные данные или задачу в готовый результат.

4. Контрольные вопросы

1. Что такое репозиторий и какие задачи он решает?
2. Чем отличается распределённая система контроля версий от централизованной?
3. Какие каталоги обычно включаются в структуру программного проекта и почему?
4. Для чего нужен файл .gitignore? Приведите примеры шаблонов.
5. Что такое коммит? Какие требования предъявляются к сообщениям коммитов?
6. Что такое ветка в Git? Зачем нужно ветвление?
7. Как связать локальный репозиторий с удалённым?
8. Что такое конфликт слияния и как его разрешить?
9. Какие стратегии ветвления вы знаете? В чём их различия?
10. Что такое CI/CD? Приведите примеры инструментов.
11. Что такое Единый реестр российского ПО и зачем он создан?
12. Назовите известные вам российские аналоги Git. Чем они отличаются от зарубежных?
13. Какие преимущества и недостатки имеет использование отечественных систем контроля версий?

5. Критерии оценивания

Баллы	Уровень	Требования
53–79	Базовый	Выполнены все пункты уровня 1. Отчёт содержит структуру, .gitignore, README.md, историю коммитов.
80–92	Продвинутый	Выполнены пункты уровня 1 и 2. Есть удалённый репозиторий, ветка develop, слияние, разрешён конфликт, описана стратегия ветвления, проведён сравнительный анализ платформ.
93–100	Высокий	Выполнены пункты уровня 1–3. Настроен CI, созданы шаблоны Issue/PR, оформлена документация, создан тег, проведено ревью, подготовлен обзор российских аналогов Git. Отчёт оформлен аккуратно, без ошибок.

Примечание: при неполном выполнении требований конкретного уровня заданий оценка снижается.

6. Рекомендуемая литература

1. Документация Git — <https://git-scm.com/doc>
2. GitHub Docs — <https://docs.github.com/>
3. GitLab Documentation — <https://docs.gitlab.com/>
4. Chacon S., Straub B. Pro Git. — Apress, 2014.
5. Официальная документация GitHub Actions — <https://docs.github.com/en/actions>
6. Стандарт оформления README — <https://www.makeareadme.com/>
7. Единый реестр российского программного обеспечения — <https://reestr.digital.gov.ru/>
8. Официальный сайт GitFlic — <https://gitflic.ru/>
9. Официальный сайт GitFlame — <https://gitflame.ru/>
10. Официальный сайт Gitlife — <https://gitlife.ru/>
11. Официальный сайт Deckhouse — <https://deckhouse.ru/>
12. Официальный сайт EvaTeam — <https://evateam.ru/>
13. Mos.Hub — <https://hub.mos.ru/>

7. Методические указания

Все команды Git выполняются в терминале (Git Bash для Windows).

Чтобы открыть терминал:

Windows: нажмите правой кнопкой мыши в папке проекта → «Git Bash Here» (если установлен Git) или откройте Git Bash из меню «Пуск» и перейдите в нужный каталог командой `cd`.

Linux: `Ctrl+Alt+T`.

macOS: Applications → Utilities → Terminal.

Перед началом работы убедитесь, что Git установлен: введите в терминале `git --version`. Если команда не найдена, установите Git с официального сайта.

При работе с удалённым репозиторием используйте личный аккаунт на выбранной платформе (GitHub, GitLab, GitFlic, GitFlame и др.).

При выборе удалённой платформы студент может использовать как зарубежные (GitHub, GitLab), так и российские (GitFlic, GitFlame, Gitlife Enterprise, Mos.Hub) сервисы. При использовании российских платформ необходимо учитывать особенности их интерфейса и доступные функции.

Не забывайте делать коммиты после каждого логически завершённого этапа⁵.

Сообщения коммитов должны быть краткими и отражать суть изменений (например, Add README.md, Configure .gitignore, Implement input module).

Для редактирования файлов можно использовать любой текстовый редактор (nano, vim, Notepad++, VS Code). В Git Bash доступен nano.

⁵ Коммит в Git — это моментальный снимок состояния вашего проекта в определенный момент времени, который сохраняет изменения и историю.